



Manipulación y Gestión de Datos

Clase 3 - Creación y manipulación de tablas (Parte III)

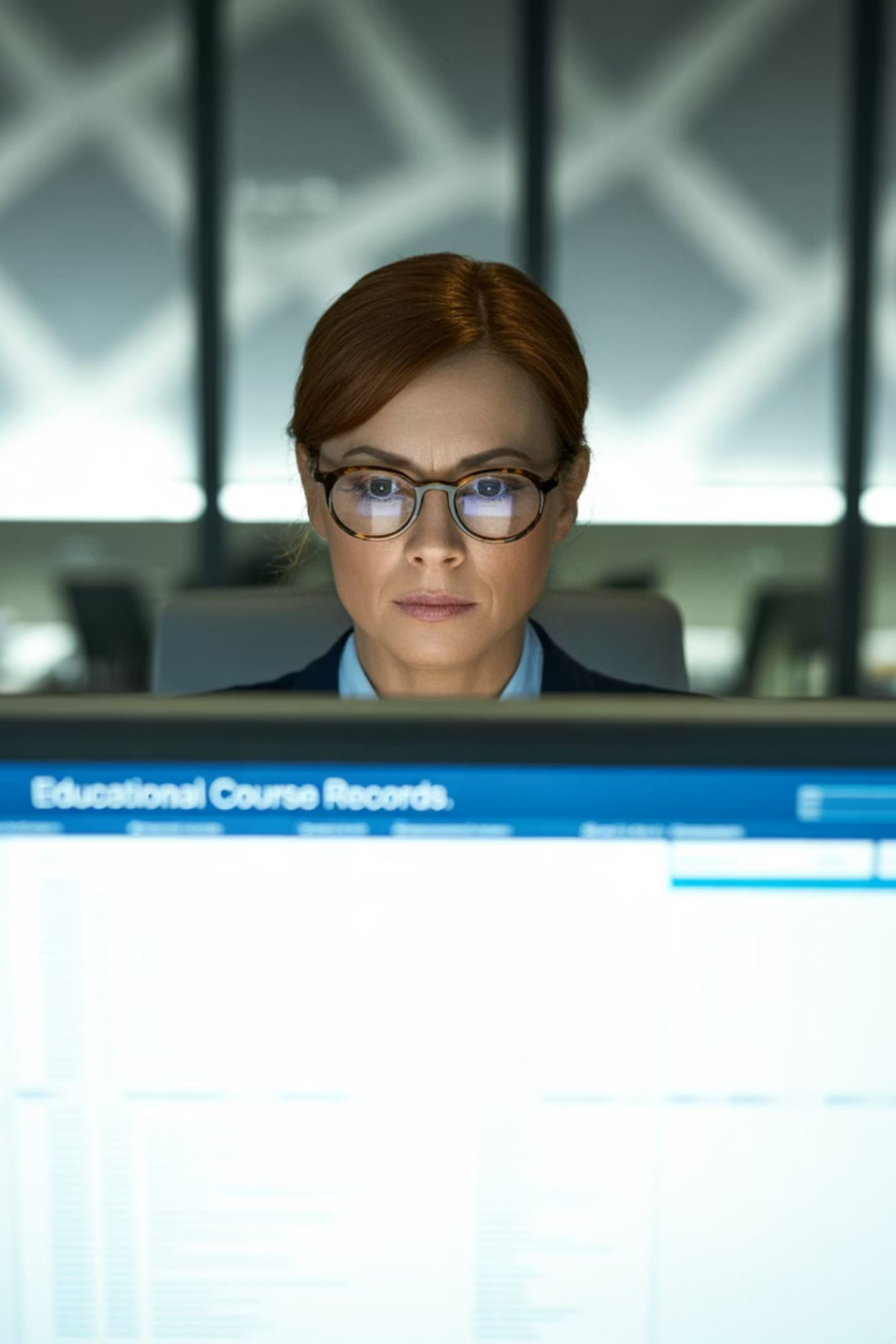




¿Qué aprenderás en esta sesión?

Aprenderemos a actualizar registros en una tabla relacional utilizando la instrucción UPDATE, incorporando condiciones que aseguren la modificación precisa y controlada de los datos.

Veremos cómo aplicar esta operación considerando restricciones existentes, validación previa de condiciones, y evitando efectos no deseados sobre los datos. Abordaremos también buenas prácticas y errores comunes en contextos productivos.



Desafío inicial

Formas parte del equipo de soporte de datos de una institución educativa que ha finalizado el registro de cursos y participantes en su base de datos. Durante la revisión rutinaria, se detecta que algunos registros presentan errores como duraciones incorrectas en cursos, nombres mal escritos o modalidades inconsistentes. Se te ha encomendado corregir estos registros manualmente, asegurando que los cambios sean precisos, que no se alteren registros no deseados y que se respeten todas las restricciones de la base de datos, como claves primarias, NOT NULL, CHECK, y valores únicos.

¿Cómo puedes actualizar de forma segura y controlada los registros existentes en una tabla relacional sin afectar la integridad del resto de los datos ni violar las restricciones del modelo de datos?

Durante esta sesión, aprenderás a utilizar correctamente la instrucción UPDATE junto con la cláusula WHERE para modificar registros específicos.

Aplicarás validaciones previas, revisarás buenas prácticas y realizarás ejemplos de actualización basados en errores comunes encontrados en ambientes productivos. Al finalizar, serás capaz de ejecutar actualizaciones controladas sin comprometer la consistencia de la información.

Actualización de valores existentes en una tabla

¿Qué aspectos debes considerar antes de modificar registros ya almacenados en una base de datos relacional?

Actualizar información en una base de datos es una operación cotidiana pero crítica. En entornos productivos, modificar un dato de forma incorrecta o sin los controles adecuados puede provocar pérdidas de integridad, errores de consistencia y, en casos graves, impactos operacionales. Por ello, es esencial comprender tanto la sintaxis como las buenas prácticas asociadas a la instrucción UPDATE, que permite modificar uno o varios valores en registros ya existentes.

Sintaxis básica de UPDATE

La instrucción UPDATE se utiliza para modificar datos existentes en una tabla. Su sintaxis básica es:

```
UPDATE nombre_tabla  
SET columna1 = valor1 [, columna2 = valor2, ...]  
WHERE condición;
```

- nombre_tabla: tabla donde se realizará la actualización.
- SET: establece los valores nuevos para las columnas especificadas.
- WHERE: filtra los registros a modificar; si se omite, se actualizan todos los registros de la tabla.

Ejemplo:

```
UPDATE cursos
SET duracion = 30
WHERE nombre_curso = 'SQL aplicado a negocios';
```

Este comando actualiza la duración del curso "SQL aplicado a negocios" a 30.

Además de la sintaxis básica, UPDATE puede combinarse con subconsultas o funciones para calcular los nuevos valores. Esto permite aplicar lógica condicional y transformaciones dinámicas, como en el siguiente ejemplo:

```
UPDATE cursos
SET duracion = duracion + 10
WHERE modalidad = 'Presencial';
```

Este código incrementa la duración de todos los cursos presenciales en 10 horas, lo que puede ser útil ante un rediseño curricular.

Esta capacidad es clave en procesos como ajustes masivos de precios, extensiones de duración de proyectos o actualizaciones periódicas de datos operativos.

Errores comunes y buenas prácticas

Errores comunes:

- Usar operadores incorrectos (= en lugar de +=).
- Olvidar colocar comas entre columnas en el bloque SET.

Buenas prácticas:

- Usar nombres explícitos de columnas para claridad.
- Documentar actualizaciones masivas, especialmente en entornos colaborativos o productivos.

Rol crítico de la cláusula WHERE

El uso adecuado de WHERE es fundamental. Omitir esta cláusula puede causar la actualización de todos los registros de la tabla, lo que puede ser desastroso en un entorno real.

❌ Ejemplo incorrecto:

```
UPDATE cursos
SET modalidad = 'Presencial';
-- ¡Actualiza todos los cursos sin filtrar!
```

Buenas prácticas:

- Siempre verificar qué registros serán afectados antes de ejecutar el UPDATE.
- Usar una consulta SELECT con la misma condición WHERE para revisar los datos previos.

```
SELECT *  
FROM cursos  
WHERE modalidad = 'Online';
```

La cláusula WHERE acepta expresiones lógicas complejas, combinando múltiples condiciones con AND, OR y NOT, así como operadores relacionales y funciones. Por ejemplo:

```
UPDATE cursos  
SET modalidad = 'Presencial'  
WHERE duracion > 25  
    AND modalidad = 'Online';
```

En áreas como logística o ventas, esto permite hacer actualizaciones dirigidas a subconjuntos específicos, como productos con baja rotación o clientes con estados especiales.

Errores comunes:

- Usar = en lugar de LIKE para valores parciales.
- No colocar paréntesis en expresiones lógicas compuestas.

Actualización de múltiples columnas

Se pueden modificar varias columnas a la vez dentro de la misma sentencia:

```
UPDATE cursos
SET duracion = 40,
    modalidad = 'Presencial'
WHERE nombre_curso = 'Power BI para análisis';
```

Esto permite realizar correcciones complejas en una sola operación.

La actualización simultánea de múltiples campos es útil para asegurar consistencia en datos relacionados. Por ejemplo, si un curso cambia su modalidad de enseñanza, es posible que también deba ajustarse su duración, el tipo de sala asignada o la cantidad de participantes admitidos.

En bases de datos de gestión académica, se puede actualizar la duración, modalidad y fecha de inicio de un curso en una sola instrucción, simplificando procesos y reduciendo errores manuales:

```
UPDATE cursos
SET duracion = 45,
    modalidad = 'Online',
    fecha_inicio = '2025-08-01'
WHERE nombre_curso = 'Fundamentos de Ciencia de Datos';
```

Errores comunes y buenas prácticas en actualizaciones múltiples

Errores comunes:

- Repetir la palabra SET antes de cada columna (es incorrecto).
- Omitir comas entre los pares columna = valor.
- Introducir valores que violen restricciones de tipo de dato o longitud.

Buenas prácticas:

- Validar que los cambios en múltiples campos sean coherentes entre sí.
- Comprobar que la condición WHERE sea suficientemente específica para evitar modificaciones no deseadas.

Restricciones y validaciones durante la actualización

Cuando se realiza un UPDATE, la base de datos valida automáticamente las restricciones definidas en la tabla (NOT NULL, CHECK, UNIQUE, claves primarias y foráneas). Si una actualización viola alguna de estas restricciones, el sistema rechazará la operación.



Ejemplo de error:

```
UPDATE cursos
SET modalidad = 'Virtual'
WHERE id_curso = 2;
-- Error: modalidad no permitida por restricción CHECK
```

Tipos de restricciones en actualizaciones

Existen distintos tipos de restricciones que pueden impedir la ejecución de una actualización:

Tipo de restricción	Descripción	Resultado si se viola
NOT NULL	No permite valores nulos	Error de integridad
CHECK	Evalúa una condición lógica	Error de validación
UNIQUE	Impide duplicados	Violación de unicidad
Clave primaria	Identificador único por fila	Error de clave duplicada
Clave foránea	Relación con otra tabla	Violación de integridad referencial

Cuando se actualiza un registro relacionado con otras tablas (por ejemplo, cambiar el id_profesor de un curso), puede romperse la relación si el nuevo valor no existe en la tabla referenciada. Esto es común en bases de datos académicas, bancarias y logísticas.

Errores comunes y buenas prácticas con restricciones

Errores comunes:

- Introducir valores vacíos en campos NOT NULL.
- Romper relaciones con claves foráneas.
- Asignar valores duplicados en campos con restricción UNIQUE.
- Ejecutar un UPDATE sin WHERE, afectando toda la tabla.

Buenas prácticas:

- Consultar la estructura de la tabla (\d nombre_tabla en PostgreSQL o DESCRIBE en MySQL) antes de hacer cambios.
- Validar previamente los datos que se desean modificar.
- Probar las actualizaciones primero en un entorno de prueba.
- Utilizar transacciones para aplicar cambios solo si todas las condiciones se cumplen.

Aplicaciones en contextos productivos

En ambientes reales, las actualizaciones se utilizan para:



Corregir errores de captura o migración de datos.



Actualizar información obsoleta (por ejemplo, cambios de modalidad o de duración de un curso).



Implementar decisiones operativas (por ejemplo, cambios de estado, cierres de proceso, etc.).

Ejemplo aplicado:

En una empresa de capacitación, tras revisar los registros del sistema, se identifica que todos los cursos registrados en modalidad "Presencial" deben cambiarse a "Online" por cambios sanitarios:

```
UPDATE cursos
SET modalidad = 'Online'
WHERE modalidad = 'Presencial';
```

Dominar la instrucción UPDATE permite realizar modificaciones eficientes y seguras en las bases de datos. En entornos profesionales, estas operaciones son frecuentes y deben ejecutarse con total comprensión de sus implicancias.

Actividad guiada:

Corrigiendo registros de cursos en una base académica



Objetivo de la actividad

Aplicar actualizaciones controladas de datos en una base relacional, utilizando correctamente la instrucción UPDATE y la cláusula WHERE, validando las restricciones existentes y previniendo modificaciones erróneas o masivas.

Situación problema

Eres parte del equipo de soporte técnico de una institución académica que implementó recientemente una base de datos de cursos.

Al revisar la información, se detectan inconsistencias: algunas duraciones de cursos están mal ingresadas, y ciertas modalidades están escritas de forma errónea.

Tu tarea es aplicar las actualizaciones necesarias, sin afectar otros datos y respetando las restricciones establecidas en la base.



Instrucciones

Crea la siguiente tabla con datos:

```
CREATE TABLE cursos (  
  id_curso INT PRIMARY KEY,  
  nombre_curso VARCHAR(50) NOT NULL,  
  duracion INT CHECK (duracion > 0),  
  modalidad VARCHAR(20) CHECK (modalidad IN ('Online', 'Presencial'))  
);  
  
INSERT INTO cursos (id_curso, nombre_curso, duracion, modalidad)  
VALUES  
  (1, 'Excel básico', 20, 'Presencial'),  
  (2, 'Análisis de datos', 25, 'Presencial'),  
  (3, 'SQL aplicado a negocios', 10, 'Presencial'),  
  (4, 'Python para principiantes', 30, 'Online'); -- error ortográfico
```



Pasos para la actividad guiada

1. Identifica errores de modalidad y duración.
2. Aplica actualizaciones específicas:

```
UPDATE cursos
SET modalidad = 'Online'
WHERE id_curso = 4;

UPDATE cursos
SET duracion = 30
WHERE nombre_curso = 'SQL aplicado a negocios';
```

Verifica los cambios ejecutando:

```
SELECT * FROM cursos;
```

Reflexiona: ¿qué precauciones se tomaron?, ¿qué errores se podrían haber cometido si no se usara WHERE?

Materiales necesarios

- Editor SQL (pgAdmin, DBeaver, MySQL Workbench o similar)
- Código de creación de tabla y datos proporcionado por el relator

{desafío}
latam_



Actividad práctica autónoma:

**Corrección de datos erróneos
en una tabla de programas
académicos**



Objetivo de la actividad

Aplicar la instrucción UPDATE de manera autónoma para corregir registros erróneos en una base de datos simulada del entorno educativo, validando el cumplimiento de las restricciones establecidas y justificando cada actualización realizada.

Contexto

Formas parte del equipo de datos de un centro de formación técnica. Se identificaron errores en una tabla que registra información de programas académicos.

Algunos valores de duración están desactualizados, y hay errores tipográficos en la modalidad.

Tu tarea es identificar y corregir estos errores de manera autónoma, garantizando la integridad de la información.



Instrucciones

Crea la siguiente tabla y sus registros iniciales:

```
CREATE TABLE programas (  
  id_programa INT PRIMARY KEY,  
  nombre_programa VARCHAR(50) NOT NULL,  
  duracion INT CHECK (duracion > 0),  
  modalidad VARCHAR(20) CHECK (modalidad IN ('Online', 'Presencial'))  
);  
  
INSERT INTO programas (id_programa, nombre_programa, duracion, modalidad)  
VALUES  
  (1, 'Data Science Inicial', 60, 'Presencial'),  
  (2, 'Inteligencia Artificial', 40, 'Presencial'),  
  (3, 'Big Data', 15, 'Presencial'),  
  (4, 'Análisis de Datos con Python', 45, 'Onlne'); -- error tipográfico
```


Pasos para la actividad

1 - Verifica el contenido actual:

```
SELECT * FROM programas;
```

Realiza las actualizaciones necesarias:

- Corregir la modalidad del programa con error tipográfico.
- Ajustar la duración del programa 'Big Data' a 30 horas.

2 - Ejecuta las sentencias:

```
UPDATE programas
SET modalidad = 'Online'
WHERE id_programa = 4;

UPDATE programas
SET duracion = 30
WHERE nombre_programa = 'Big Data';
```

Registra tus decisiones en la siguiente tabla:

Campo actualizado	Valor corregido	Justificación
-------------------	-----------------	---------------



Resumen



Comprendimos cómo funciona la instrucción UPDATE en SQL para modificar registros existentes en una tabla.



Aprendimos a utilizar correctamente la cláusula WHERE para evitar actualizaciones masivas no deseadas.



Identificamos buenas prácticas y errores comunes al realizar actualizaciones sobre bases de datos con restricciones.



Aplicamos validaciones antes y después de actualizar datos para garantizar integridad.



Realizamos actualizaciones sobre ejemplos aplicables a contextos educativos reales.



Próxima sesión...

Nos adentraremos en la creación y manipulación de tablas, donde abordaremos la eliminación de registros específicos utilizando la instrucción DELETE.

Exploraremos cómo eliminar datos de forma segura y selectiva, y qué precauciones deben tomarse para evitar pérdidas de información valiosa.

Además, analizaremos los peligros asociados a la eliminación de registros, tanto desde el punto de vista técnico como organizacional.

Preguntas de cierre

1

¿Qué componentes debe incluir toda instrucción UPDATE para ser segura?

2

¿Qué rol cumple la cláusula WHERE al actualizar datos?

3

¿Cómo validarías que un UPDATE no ha violado restricciones en la base de datos?

4

¿En qué casos una actualización puede ser rechazada por el sistema?

5

¿Qué estrategias aplicarías para prevenir errores al modificar información productiva?

Referencias bibliográficas

- Coronel, C., & Morris, S. (2020). Database Systems: Design, Implementation, & Management (13th ed.). Cengage Learning.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.
- W3Schools. (2024). SQL UPDATE Statement. Recuperado de https://www.w3schools.com/sql/sql_update.asp
- Oracle. (2023). SQL Language Reference - UPDATE. Recuperado de <https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/UPDATE.html>
- PostgreSQL Documentation. (2024). SQL Commands: UPDATE. Recuperado de <https://www.postgresql.org/docs/current/sql-update.html>